WONBET · FASE 1 DE 3 — GUÍA DE PREGUNTAS

Lo que probablemente te van a preguntar

20 preguntas organizadas por tema, con la respuesta corta que darías en el momento — no para leer de memoria, sino para tener el argumento claro. Las marcadas en ámbar son las "trampa": las que buscan hacerte dudar de una decisión que sí fue a propósito.

01 Arquitectura general 02 Flujo del JWT 03 Comunicación interna 04 Base de datos
05 Decisiones de diseño 06 Manejo de errores 07 Preguntas "¿y si...?"

01 · ARQUITECTURA GENERAL

Q. ¿Por qué dividiste WonBet en microservicios en vez de un solo servidor?

Para separar responsabilidades: usuarios, eventos y apuestas son dominios distintos, cada uno con su propio servidor, su propia base de datos y su propio ciclo de vida. Un gateway al frente es el único punto de entrada para el cliente.

Q. ¿Qué hace exactamente el API Gateway?

Nada más que enrutar. Es un proxy puro con `http-proxy-middleware`: recibe la petición en el puerto `3000` y la reenvía tal cual —headers incluidos— al microservicio correcto según la URL. No toca ni lee el token.

Q trampa. Si el gateway no verifica nada, ¿no es un hueco de seguridad?

No: la seguridad no depende del gateway, depende de que cada microservicio verifique el JWT por su cuenta antes de hacer cualquier cosa. El gateway es solo tránsito — si alguien le pega directo a un microservicio saltándose el gateway, igual le van a pedir un token válido.

02 · FLUJO DEL JWT

Q. ¿Cómo se genera el JWT?

`ms-usuarios` lo firma en `/login`, después de validar la contraseña con bcrypt, con `jwt.sign({correo, rol}, JWT_SECRET, {expiresIn:'2h', algorithm:'HS256'})`.

Q. ¿Por qué los tres servicios pueden verificar un token que no generaron ellos?

Porque comparten la misma clave secreta (`JWT_SECRET`) hardcodeada en cada `middleware/auth.js` . Con HMAC (HS256), cualquiera que tenga la misma clave puede verificar la firma, sin preguntarle al que la firmó.

Q trampa. Si ya sabías que ibas a usar HS256, ¿para qué especificar `algorithms: ['HS256']` en `verify` ?

Es la recomendación oficial de `jsonwebtoken` para evitar un ataque de "confusión de algoritmo", donde alguien manipula el token para forzar que el servidor lo valide con un algoritmo distinto. Restringirlo explícitamente cierra esa puerta, aunque acá nunca se use otro algoritmo.

Q. ¿Qué pasa si el token ya expiró?

`jwt.verify` lanza un error, el middleware lo atrapa y responde `401` . El usuario tendría que volver a hacer login para conseguir uno nuevo.

Q. ¿Dónde se guarda el token en el cliente?

En `localStorage` del navegador. El frontend lo manda en cada petición como header `Authorization: Bearer <token>` .

03 · COMUNICACIÓN ENTRE MICROSERVICIOS

Q. ¿Cómo sabe `ms-apuestas` la cuota y el estado de un evento?

Le pega directo a `ms-eventos` , sin pasar por el gateway: `GET http://localhost:3002/internal/eventos/:id` . Es una llamada HTTP normal entre dos procesos Node en la misma máquina.

Q. ¿Por qué ese endpoint interno no pide token?

Porque es tráfico servicio-a-servicio, no de un usuario final. Igual que en el ejemplo del profesor (`ms-compras → ms-productos`), se asume que solo otro microservicio de confianza en la misma red conoce esa ruta.

Q trampa. ¿No es inseguro que cualquiera pueda llamar a ese endpoint interno sin autenticarse?

En producción real sí sería un riesgo — normalmente se resuelve con una red privada donde ese puerto no es alcanzable desde afuera, o con una clave interna compartida. Acá se dejó abierto a propósito, por simplicidad pedagógica, igual que el ejemplo de referencia.

04 · BASE DE DATOS

Q. ¿Por qué SQLite y no PostgreSQL?

Para bajar la infraestructura al mínimo: SQLite es un solo archivo, no necesita un servidor de base de datos aparte ni Docker. Cada microservicio tiene su propio archivo `.db`, independiente de los demás.

Q. ¿Qué es `node:sqlite` ? ¿Es lo mismo que `better-sqlite3` ?

Es el módulo de SQLite incluido en Node.js desde la versión 22 — no hay que instalar nada. Se usa casi igual (`db.prepare(sql).get/all/run()`), pero no requiere compilar un módulo nativo. Se eligió en vez de `better-sqlite3` porque ese paquete falla al compilar en Node 26 por incompatibilidades con V8.

Q. ¿Las bases de datos comparten información entre sí?

No, cada una es completamente independiente. Si `ms-apuestas` necesita datos de un evento, no lee la base de `ms-eventos` directamente — se lo pide por HTTP. Esa es la idea central de microservicios: cada uno es dueño exclusivo de sus datos.

05 · DECISIONES DE DISEÑO

Q trampa. ¿Por qué "Mis apuestas" muestra las de todos los usuarios, no solo las mías?

Es deliberado, para mantener la misma simplicidad que el ejemplo de referencia (`ms-compras` tampoco filtra por usuario). No es un descuido: filtrar sería una línea más (`WHERE usuario_correo = ?`), pero se dejó así a propósito.

Q. ¿Por qué no hay roles ni permisos (admin vs. usuario)?

Porque el ejemplo que se sigue tampoco los tiene, y agregar autorización por rol sumaría una capa de complejidad que no aporta al objetivo del proyecto. El campo `rol` existe en el modelo, pero todavía no restringe nada.

Q trampa. ¿Este proyecto está listo para producción?

No, y no pretende estarlo. Es una versión educativa: SQLite en vez de una base con réplicas, un secreto hardcodeado en vez de un vault, un endpoint interno sin autenticación. Cada simplificación es un punto consciente, no un error que no vimos.

06 · MANEJO DE ERRORES

Q. ¿Qué pasa si dos personas se registran con el mismo correo exactamente al mismo tiempo?

Los dos pueden pasar la verificación de "¿ya existe?" antes de que cualquiera inserte — es una condición de carrera. Cuando el segundo `INSERT` choca con la restricción `UNIQUE` de SQLite, se atrapa ese error específico y se devuelve un `409` limpio, en vez de que la petición se cuelgue.

Q. ¿Por qué es tan importante el `try/catch` en los handlers `async`?

Porque el proyecto usa Express 4, y a diferencia de Express 5, no reenvía automáticamente los errores de una función `async`. Sin `try/catch`, un error ahí deja la petición colgada para siempre en vez de responder un error controlado.

Q. ¿Qué pasa si mando un monto no numérico al apostar?

Se valida con `Number.isFinite()` antes de tocar la base — si no es válido, responde `400` con un mensaje claro. Sin esa validación, el valor llegaba hasta SQLite y volvía como `500`, lo cual es engañoso porque el problema era del cliente, no del servidor.

07 · PREGUNTAS "¿Y SI...?"

Q. ¿Cómo agregarías un cuarto microservicio, por ejemplo notificaciones?

Crearía `ms-notificaciones/` con el mismo patrón: su propio `package.json`, su servidor Express, su `middleware/auth.js` con el mismo `JWT_SECRET`, su base SQLite propia, y una ruta nueva en el gateway.

Q. ¿Qué pasa si `ms-eventos` se cae mientras alguien está apostando?

El `fetch` desde `ms-apuestas` falla, el `try/catch` dedicado a esa llamada lo atrapa, y responde `503` avisando que no se pudo conectar con Eventos — no se guarda ninguna apuesta a medias.

Q. Si tuvieras que llevarlo a producción real, ¿qué cambiarías primero?

Sacaría el `JWT_SECRET` del código a una variable de entorno real, pondría el endpoint interno detrás de una red privada, cambiaría SQLite por una base con concurrencia real, y filtraría "mis apuestas" por usuario.

Fase 1 lista. Siguiente: **Fase 2** — practicamos en vivo, te hago preguntas de esta guía (y alguna nueva) en el chat para que las respondas en tus palabras. Cuando terminemos esa ronda armamos la **Fase 3** (el cheat sheet de una página) con lo que más te costó.